# OFSD FUND VALUATION SYSTEM

## Development Guide

Migration: Percentage-Based to Share-Based (NAV) Valuation

---

Date: April 17, 2026
Timeline: April 18-21, 2026 (Weekend + Monday)
Audience: Frontend Dev, Backend Dev
**CONFIDENTIAL**

# 1. Executive Summary

This document provides the complete development plan for migrating the OFSD fund valuation system from a percentage-based compounding model to a share-based (NAV per share) valuation model. This migration aligns the system with AXYS (Mauritius) methodology.

## 1.1 The Problem

The current system calculates investor returns by applying a percentage performance rate over a period and distributing gains proportionally by capital weight. AXYS operates on a unitised (share-based) model where each investor owns shares purchased at the NAV per share on their deployment date, and their position value is simply shares multiplied by the current NAV per share.

When clients join mid-period, the two models diverge: percentage proration gives different results than share-based compounding. The share-based model is the contractually correct one.

## 1.2 The Solution

Replace the valuation engine entirely. Every investor position becomes: shares (bought at deployment NAV) multiplied by current NAV per share. Performance is derived from NAV movement, not input as a percentage. Reconciliation checks total system NAV against AXYS figures.

## 1.3 What Changes

| Component | Current State | New State |
|---|---|---|
| Valuation Model | Percentage-based compounding | Share-based: shares x current NAV |
| Client Entry | Excel batch upload | Individual client entry, auto-batching |
| Batch Purpose | Calculation unit | Operational grouping + reconciliation |
| Fund Structure | CoreFund (name only) | Fund > Classes (with currency, NAV history) |
| Data Model | No share tracking | Shares per investor, NAV history per class |
| Statements | Gain/profit columns | NAV-based: shares, price, market value |
| Dashboard Charts | Combined KES+USD pie chart | Separate KES and USD portfolios |
| Reconciliation | AXYS total comparison | NAV x total shares = AXYS NAV |

## 1.4 Timeline

| Day | Frontend Dev | Backend Dev |
|---|---|---|
| Sat Apr 18 (AM) | Fund/Class management pages | Database migration + new schema |
| Sat Apr 18 (PM) | Redesign Add Investor flow | Fund/Class/ShareClass API endpoints |
| Sun Apr 19 (AM) | Batch redesign (auto-batch, tracker) | Batch auto-creation + close/export logic |
| Sun Apr 19 (PM) | Deployment popup + share display | Deployment endpoint (share calculation) |
| Mon Apr 20 (AM) | Valuation page redesign | New valuation engine (NAV-based) |
| Mon Apr 20 (PM) | Statement templates + dashboard | Statement generation + reconciliation |

| | | |
|---|---|---|
| Mon Apr 20 (Eve) | Integration testing | Integration testing |

## 1.5 Parallel Work Strategy

Frontend builds first. Backend tests against the real UI. The two streams connect at API contracts defined in this document. Both devs must use the exact field names, endpoint paths, and data shapes specified here.

**CRITICAL RULE**

Each class can only have ONE currency type. KES classes are KES only. USD classes are USD only. A fund can have classes in both currencies, but a single class is always one currency. Never mix currencies within a class.

# 2. Data Model (New Schema)

The new schema introduces share classes, NAV tracking, and individual share ownership. All new tables are additive; existing tables are modified but not dropped.

## 2.1 New Tables

### 2.1.1 share_classes

Each fund has one or more share classes. Each class has exactly one currency.

| Column | Type | Constraints | Description |
|---|---|---|---|
| id | Integer | PK, auto | |
| core_fund_id | Integer | FK core_funds.id, NOT NULL | Parent fund |
| class_name | String(100) | NOT NULL | e.g. 'Class I Participating Shares' |
| class_code | String(20) | NOT NULL, UNIQUE | e.g. 'KES_I', 'USD_I', 'USD_R' |
| currency | String(3) | NOT NULL, CHECK IN ('KES','USD') | Single currency per class |
| is_active | Boolean | DEFAULT true | |
| created_at | DateTime(tz) | NOT NULL | |

### 2.1.2 nav_history

Stores NAV per share for each class at each valuation date. This is the core pricing table.

| Column | Type | Constraints | Description |
|---|---|---|---|
| id | Integer | PK, auto | |
| share_class_id | Integer | FK share_classes.id, NOT NULL | Which class |
| nav_date | Date | NOT NULL | Valuation/deployment date |
| nav_per_share | Numeric(20,6) | NOT NULL | Price per share on this date |
| total_fund_nav | Numeric(20,2) | NULLABLE | AXYS total (for recon) |
| source | String(20) | NOT NULL | deployment or valuation |
| created_at | DateTime(tz) | NOT NULL | |

**UNIQUE CONSTRAINT**

nav_history has a unique constraint on (share_class_id, nav_date, source) to prevent duplicate entries for the same class on the same date from the same source type.

### 2.1.3 investor_shares

Records how many shares each investor holds in each class, linked to the batch they entered through.

| Column | Type | Constraints | Description |
|---|---|---|---|
| id | Integer | PK, auto | |
| investment_id | Integer | FK investments.id, NOT NULL | Link to investment record |
| share_class_id | Integer | FK share_classes.id, NOT NULL | Which class |
| batch_id | Integer | FK batches.id, NOT NULL | Which batch (for recon) |
| shares_owned | Numeric(20,6) | NOT NULL | Number of shares |
| deployment_nav | Numeric(20,6) | NOT NULL | NAV per share at purchase |
| deployment_date | Date | NOT NULL | Date shares were purchased |
| is_active | Boolean | DEFAULT true | False after full withdrawal |
| created_at | DateTime(tz) | NOT NULL | |

### 2.1.4 valuation_snapshots

Stores the result of each monthly valuation for audit trail and statement generation.

| Column | Type | Constraints | Description |
|---|---|---|---|
| id | Integer | PK, auto | |
| investor_shares_id | Integer | FK investor_shares.id, NOT NULL | Which holding |
| valuation_date | Date | NOT NULL | Date of this valuation |
| nav_per_share | Numeric(20,6) | NOT NULL | NAV used for this valuation |
| shares_held | Numeric(20,6) | NOT NULL | Shares at time of valuation |
| market_value | Numeric(20,2) | NOT NULL | shares_held x nav_per_share |
| performance_pct | Numeric(10,4) | NOT NULL | Period performance percentage |
| created_at | DateTime(tz) | NOT NULL | |

## 2.2 Modified Tables

### 2.2.1 batches (modified)

Add columns to support auto-batching, class linkage, and the new workflow:

| New Column | Type | Description |
|---|---|---|
| share_class_id | Integer FK | Links batch to one specific share class |
| auto_close_at | DateTime(tz) | Thursday 9:00 AM auto-close timestamp |
| is_open | Boolean DEFAULT true | False once batch is closed for new investors |
| exported_at | DateTime(tz) | When the batch Excel was exported |
| total_shares | Numeric(20,6) | Sum of all investor shares in this batch |
| deployment_nav | Numeric(20,6) | NAV per share at batch deployment |

### 2.2.2 investments (modified)

The existing investment table stays but gains share-related fields:

| New Column | Type | Description |
|---|---|---|
| share_class_id | Integer FK | Which share class this investment is in |
| deposit_date | Date | Client deposit date (distinct from date_deposited which is system entry date) |

**MIGRATION NOTE**

Existing data will need to be backloaded. Use current NAV statements to calculate historical share counts for existing investors. The formula is: shares = net_invested_amount / deployment_nav_per_share. This is a one-time data migration task.

# 3. API Contract

These are the exact endpoints, request/response shapes, and field names that both frontend and backend must use. Any deviation breaks integration.

## 3.1 Fund & Class Management

### POST /api/v1/funds

Create a new fund.

```
Request: { "fund_name": "Axiom", "fund_code": "AXM" }
Response: { "status": 201, "data": { "id": 1, "fund_name": "Axiom", "fund_code":
"AXM", "is_active": true } }
```

### POST /api/v1/funds/:fund_id/classes

Add a share class to a fund. Each class has exactly one currency.

```
Request: { "class_name": "Class I Participating Shares", "class_code": "KES_I",
"currency": "KES" }
Response: { "status": 201, "data": { "id": 1, "core_fund_id": 1, "class_name":
"...", "class_code": "KES_I", "currency": "KES" } }
```

### GET /api/v1/funds/:fund_id/classes

List all share classes for a fund.

```
Response: { "status": 200, "data": [{ "id": 1, "class_name": "...", "class_code":
"KES_I", "currency": "KES" }, ...] }
```

## 3.2 Investor Management

### POST /api/v1/investors

Add a new investor. System auto-assigns to open batch or creates new one.

```
Request: {
  "investor_name": "Jane Doe", "investor_email": "jane@example.com",
  "investor_phone": "+254700000000", "internal_client_code": "INV-001",
  "fund_id": 1, "share_class_id": 1,
  "deposit_amount": 1000000.00, "deposit_date": "2026-04-15"
}
Response: { "status": 201, "data": {
  "investment_id": 42, "batch_id": 7, "batch_name": "auto-generated",
  "fund_name": "Axiom", "class_code": "KES_I", "currency": "KES",
  "deposit_amount": 1000000.00
} }
```

**AUTO-BATCH LOGIC**

Backend checks: is there an open batch for this share_class_id where is_open=true and auto_close_at > now()? If yes, add investor to that batch. If no, create a new batch with auto_close_at set to next Thursday 9:00 AM EAT. Batch naming: '{fund_code}_{class_code}_{close_date}' e.g. 'AXM_KES_I_2026-04-17'.

## POST /api/v1/investors/existing

Add a new deposit for an existing investor (identified by client_code).

```
Request: {
  "internal_client_code": "INV-001",
  "fund_id": 1, "share_class_id": 1,
  "deposit_amount": 500000.00, "deposit_date": "2026-04-16"
}
```

## 3.3 Batch Operations

### POST /api/v1/batches/:batch_id/close

Close batch and generate export Excel. Triggered automatically at Thursday 9 AM or manually.

```
Response: { "status": 200, "data": { "batch_id": 7, "is_open": false,
"exported_at": "...",
  "export_url": "/api/v1/batches/7/export", "investors_count": 5,
"total_deposited": 5000000 } }
```

### GET /api/v1/batches/:batch_id/export

Download the batch Excel for Mauritius. Named: {fund}_{class}_{date}.xlsx

### PATCH /api/v1/batches/:batch_id/transfer

Mark batch as transferred. Triggers investor email (stage 2 of 3).

```
Request: { "transfer_transaction_cost": 150.00 }
```

### PATCH /api/v1/batches/:batch_id/deploy

Deploy batch. This is the most critical endpoint. Calculates shares for every investor.

```
Request: {
  "deployment_date": "2026-04-01",
  "nav_per_share": 1329.4779,
  "total_nav": 1072636.69,
  "total_shares": 771.92,
  "transfer_cost": 150.00,
  "entry_fee_percent": 2.0
}
```

**Backend calculation per investor:**

```
net_amount = deposit_amount - transfer_cost_share - (deposit_amount *
entry_fee_percent / 100)
investor_shares = net_amount / nav_per_share
```

**Validation check:**

```
sum(all investor_shares in batch) should approximately equal total_shares
sum(all investor_shares) * nav_per_share should approximately equal total_nav

Response: { "status": 200, "data": {
  "batch_id": 7, "deployment_date": "2026-04-01", "nav_per_share": 1329.4779,
  "total_shares": 771.92, "total_nav": 1072636.69,
  "investors": [
    { "client_code": "INV-001", "deposit": 1000000, "net_invested": 970000,
      "shares": 729.86, "deployment_nav": 1329.4779 },
    ...
  ],
  "reconciliation": { "calculated_total_shares": 771.92, "diff": 0.00, "status":
"PASS" }
} }
```

## 3.4 Valuation

### POST /api/v1/valuation/nav

Monthly valuation. Input current NAV per share and total fund NAV for each class.

```
Request: {
  "valuation_date": "2026-04-10",
  "classes": [
    { "share_class_id": 1, "nav_per_share": 1389.5737, "total_fund_nav": 1072636.69
},
    { "share_class_id": 2, "nav_per_share": 15.1905, "total_fund_nav": 219074.79 },
    { "share_class_id": 3, "nav_per_share": 12.9358, "total_fund_nav": 1038.81 }
  ]
}
```

**Backend process (per class):**

1. Load all active investor_shares for this class
2. Calculate system_total_nav = sum(shares_owned) * nav_per_share
3. Compare system_total_nav to total_fund_nav from AXYS
4. If difference > tolerance (e.g. 1.00 in currency): return error, do not proceed
5. If passes: for each investor, calculate market_value = shares_owned * nav_per_share
6. Calculate performance_pct = (nav_per_share - deployment_nav) / deployment_nav
7. Save valuation_snapshot for each investor
8. Save nav_history record for this class + date

```
Response: { "status": 200, "data": {
  "valuation_date": "2026-04-10",
  "classes": [{
    "class_code": "KES_I", "nav_per_share": 1389.5737,
    "system_total_nav": 1072636.96, "head_office_nav": 1072636.69,
    "difference": 0.27, "status": "PASS",
    "investors": [{ "client_code": "INV-001", "shares": 729.86,
      "market_value": 1014178.42, "performance_pct": 4.52 }, ...]
  }]
} }
```

## 3.5 Withdrawal

### POST /api/v1/withdrawals/request

```
Request: { "internal_client_code": "INV-001", "share_class_id": 1,
"shares_to_redeem": 100.00 }
```

Or: { "internal_client_code": "INV-001", "share_class_id": 1, "amount_to_withdraw": 130000.00 }

Backend converts amount to shares using current NAV if amount is given instead of shares.

### PATCH /api/v1/withdrawals/:id/confirm

```
Request: { "redemption_nav": 1389.5737, "redemption_date": "2026-04-10",
  "shares_redeemed": 100.00, "transfer_cost": 50.00, "exit_fee_applies": false }
```

If investor joined less than 6 months ago, exit_fee_applies should be true (2% exit fee).

# 4. Frontend Development Guide

The frontend dev should build all pages/components listed below. Backend dev will test against these UIs. Use the existing design system (CSS variables, component patterns). All new pages follow the same layout patterns as existing ones.

## 4.1 Fund & Class Management Page

New page at route /funds/manage. This is a one-time setup page used to create funds and their share classes.

### UI Requirements:

- Page header: 'Fund Management'
- Button: 'Add Fund' opens modal with fields: Fund Name, Fund Code
- List of existing funds as expandable cards
- Each fund card shows: fund name, number of classes, active status
- Expanded view shows classes table: Class Name, Class Code, Currency, Active
- Button per fund: 'Add Class' opens modal with fields: Class Name, Class Code, Currency (dropdown: KES or USD)
- Currency dropdown must only show KES and USD as options

**AI PROMPT: Fund Management Page**

```
Build a React page at /funds/manage for the OFSD fund valuation system. It needs: 1. A page header 'Fund Management' with an 'Add Fund' button 2. Fund list as expandable cards showing fund name, class count, status 3. Expanded view shows a table of share classes (name, code, currency, active) 4. 'Add Class' button per fund opens a modal with fields: Class Name (text), Class Code (text), Currency (dropdown with ONLY 'KES' and 'USD' as options) 5. CRITICAL: Each class can only have ONE currency. The currency dropdown must enforce this. API endpoints: GET /api/v1/funds (list), POST /api/v1/funds (create), GET /api/v1/funds/:id/classes (list classes), POST /api/v1/funds/:id/classes (add class) Use the existing design system with CSS variables (--color-brand-400, --color-bg-surface, etc.) and the same card/table patterns used on other pages.
```

## 4.2 Redesigned Add Investor Flow

Replace the current batch Excel upload with individual client entry. Route: /investors/add

### Form Fields (in order):

| Field | Type | Required | Notes |
|---|---|---|---|
| Investor Name | Text input | Yes | |
| Client Code | Text input | Yes | Unique identifier (e.g. INV-001) |
| Email | Email input | Yes | For statement delivery |
| Phone | Tel input | No | |
| Fund | Dropdown | Yes | Populated from GET /api/v1/funds |
| Share Class | Dropdown | Yes | Populated from GET /api/v1/funds/:id/classes, filtered by selected fund |
| Currency | Display only | Auto | Shows automatically based on selected class |
| Deposit Amount | Number input | Yes | Currency prefix from class selection |
| Deposit Date | Date picker | Yes | Defaults to today |

**CASCADING DROPDOWNS**

When user selects a Fund, the Share Class dropdown populates with classes for that fund. When user selects a Class, the Currency display updates automatically (e.g. selecting 'Class I KES' shows 'KES' and prefixes the amount field with 'KES'). The user never manually selects a currency.

### For Existing Investors:

If the user enters a client_code that already exists in the system, auto-populate name, email, phone from existing data. Only require: fund, class, deposit amount, deposit date. Use endpoint POST /api/v1/investors/existing.

**AI PROMPT: Add Investor Page**

```
Build a React page at /investors/add for adding new fund investors. Requirements: 1. Form with fields: Investor Name, Client Code, Email, Phone, Fund (dropdown from API), Share Class (cascading dropdown filtered by fund), Deposit Amount (with currency prefix from class), Deposit Date 2. When Fund dropdown changes, fetch classes from GET /api/v1/funds/{fundId}/classes and populate Share Class dropdown 3. When Share Class is selected, show the currency (KES or USD) as a display-only badge and as prefix on the amount field 4. Client Code field should have a debounced lookup - if code exists, auto-fill name/email/phone and switch to 'existing investor' mode (only deposit fields needed) 5. On submit: POST /api/v1/investors (new) or POST /api/v1/investors/existing (existing). Response includes batch id showing which batch they were auto-assigned to 6. Success state: Show confirmation card with investor name, batch assigned, fund/class, amount, and a note explaining the batch will close on the next Thursday at 9 AM
```

## 4.3 Batch Page Redesign

The batch list and detail pages need significant changes to reflect the new workflow.

### Batch List Page (/batches):

- Show batches grouped by status: Open, Closed (awaiting transfer), Transferred, Deployed
- Each batch card shows: batch name, fund + class, currency, investor count, total deposited, status badge, tracker stepper
- The stepper has stages: Deposit > Transfer > Deployment > Active
- Open batches show countdown to auto-close (Thursday 9 AM)

### Batch Detail Page (/batches/:id):

- Header: Batch name, fund, class, currency, status badge
- Stage stepper (Deposit > Transfer > Deployment > Active)
- Stats row: Total Deposited, Investor Count, Deployment Date (or 'Pending'), NAV at Deployment, Total Shares, Current Value
- Investors table: Name, Client Code, Deposit Amount, Net Amount (after fees), Shares, Current Value
- Action buttons based on stage:
- Stage 1 (Deposit): 'Close & Export' button
- Stage 2 (Transfer): 'Confirm Transfer' button (opens modal for transfer cost)
- Stage 3 (Deployment): 'Confirm Deployment' button (opens deployment popup)

### Deployment Popup Fields:

| Field | Type | Source |
|---|---|---|
| Deployment Date | Date picker | From Mauritius confirmation email |
| NAV per Share | Number (6 decimal places) | From Mauritius confirmation email |
| Total NAV | Number (2 decimal places) | From Mauritius confirmation email |
| Total Shares | Number (6 decimal places) | From Mauritius confirmation email |
| Transfer Cost | Number (2 decimal places) | Known at transfer stage |
| Entry Fee % | Number (display only) | Fixed at 2%, editable if changes |

**AI PROMPT: Batch Detail Deployment Popup**

```
Add a deployment confirmation modal to the batch detail page. When user clicks 'Confirm Deployment': 1. Modal opens with fields: Deployment Date (date picker), NAV per Share (number, 6 decimal places), Total NAV (number, 2 decimal places), Total Shares (number, 6 decimal places), Transfer Cost (number, pre-filled if already entered), Entry Fee % (number, default 2.0) 2. As user fills in fields, show a live preview section:    - 'Calculated Total NAV: {total shares} x {nav per share} = {result}'    - 'AXYS Total NAV: {total nav input}' - 'Difference: {diff}' with green check if < 1.00, red warning if >= 1.00 3. Submit button disabled until all fields filled AND reconciliation passes 4. On submit: PATCH /api/v1/batches/{batchId}/deploy with all field values 5. After success: refresh batch detail, show updated investor table with shares column populated
```

## 4.4 Valuation Page Redesign

Route: /valuations/create. This replaces the percentage-based valuation with NAV-based input.

### Valuation Flow:

1. User selects valuation date
2. System loads all active share classes that have deployed batches
3. For each class, user inputs: Current NAV per Share, Total Fund NAV (from AXYS)
4. System calculates: sum(all shares in class) x input NAV = calculated total
5. Compares calculated total to input total fund NAV
6. Shows reconciliation status per class (PASS/FAIL with difference)
7. Preview shows every investor: name, shares, new market value, performance %
8. User confirms. System saves valuation snapshots.

**AI PROMPT: Valuation Page**

```
Build a React page at /valuations/create for NAV-based fund valuation. Flow: 1. Date picker for valuation date at the top 2. Below that, load all active share classes from GET /api/v1/funds (then their classes) 3. For each class, show a card with: Class name, Currency, and two input fields: 'Current NAV per Share' (6 decimals) and 'Total Fund NAV' (2 decimals) 4. As user types, show live calculation: 'System shares: {total shares in system} x NAV: {input_nav} = {calculated_total}' vs 'AXYS: {input_total_nav}' with difference and PASS/FAIL badge 5. Below all class inputs, show a combined preview table: Class | Investors | Total Shares | Calculated NAV | AXYS NAV | Diff | Status 6. 'Preview Details' button calls POST /api/v1/valuation/nav with all data. Response shows per-investor breakdown 7. Investor breakdown table: Name | Client Code | Class | Shares | Market Value | Performance % 8. 'Confirm Valuation' button (only enabled if all classes PASS) commits the valuation IMPORTANT: Group the class inputs by currency. Show KES classes together, USD classes together. Never mix currencies in the same section.
```

## 4.5 Statement Templates

Three statement types need to be generated:

### 4.5.1 Deposit Statement (generated at deployment)

- Investor name, client code
- Fund name, share class, currency
- Deposit amount, transfer cost, entry fee, net invested amount
- NAV per share at deployment, number of shares purchased
- Contract note number (added later when received)

### 4.5.2 Monthly Valuation Statement

- Investor name, client code
- Fund name, share class, currency
- Shares held
- Previous NAV per share, Current NAV per share
- Performance percentage
- Current market value (shares x current NAV)
- Historical valuations table (date, NAV, value)
- Historical transactions table (deposits, withdrawals)

### 4.5.3 Withdrawal Statement

- Same header info as monthly statement
- Shares redeemed, redemption NAV, gross proceeds
- Transfer cost, exit fee (if applicable - within 6 months of joining)
- Net withdrawal amount
- Remaining shares and current value (if partial withdrawal)

## 4.6 Dashboard Changes

- Remove the combined KES+USD pie chart (cannot merge without live FX)
- Replace with two separate portfolio summary cards: one for KES total, one for USD total
- Stretch the Deposits/Withdrawals chart and Fund Performance chart to full width
- Batch table remains but shows new columns: Fund, Class, Currency, Shares

# 5. Backend Development Guide

## 5.1 Database Migration

Create migration scripts in order. Each migration should be independently reversible.

### Migration 1: Create share_classes table

**AI PROMPT: Migration 1**

```
Create an Alembic migration for the OFSD Flask backend (SQLAlchemy) that: 1. Creates table
'share_classes' with columns: id (PK), core_fund_id (FK to core_funds.id, NOT NULL),
class_name (String 100, NOT NULL), class_code (String 20, UNIQUE NOT NULL), currency (String
3, NOT NULL, CHECK IN ('KES','USD')), is_active (Boolean default True), created_at (DateTime
with tz) 2. Creates unique constraint on (core_fund_id, class_code) 3. IMPORTANT: Each class
has exactly ONE currency. The CHECK constraint must enforce this. Current CoreFund model is
in app/Batch/core_fund.py. The new model should go in app/Funds/share_class.py
```

### Migration 2: Create nav_history table

**AI PROMPT: Migration 2**

```
Create nav_history table with columns: id (PK), share_class_id (FK share_classes.id, NOT
NULL), nav_date (Date, NOT NULL), nav_per_share (Numeric 20,6, NOT NULL), total_fund_nav
(Numeric 20,2, NULLABLE), source (String 20 NOT NULL - either 'deployment' or 'valuation'),
created_at (DateTime tz). Add unique constraint on (share_class_id, nav_date, source).
```

### Migration 3: Create investor_shares table

**AI PROMPT: Migration 3**

```
Create investor_shares table: id (PK), investment_id (FK investments.id, NOT NULL),
share_class_id (FK share_classes.id, NOT NULL), batch_id (FK batches.id, NOT NULL),
shares_owned (Numeric 20,6, NOT NULL), deployment_nav (Numeric 20,6, NOT NULL),
deployment_date (Date, NOT NULL), is_active (Boolean default True), created_at (DateTime tz).
```

### Migration 4: Create valuation_snapshots table

Stores the calculated result of each monthly valuation per investor holding. This is the audit trail and the source for statement generation.

**AI PROMPT: Migration 4**

```
Create an Alembic migration for the OFSD Flask backend (SQLAlchemy) that creates the
'valuation_snapshots' table.  Columns: - id: Integer, primary key, auto-increment -
investor_shares_id: Integer, FK to investor_shares.id, NOT NULL (links to the specific
holding being valued) - valuation_date: Date, NOT NULL (the date this valuation was
performed) - nav_per_share: Numeric(20,6), NOT NULL (the NAV per share used for this
valuation) - shares_held: Numeric(20,6), NOT NULL (number of shares the investor held at time
of valuation) - market_value: Numeric(20,2), NOT NULL (calculated as shares_held x
nav_per_share) - performance_pct: Numeric(10,4), NOT NULL (percentage growth from deployment
NAV to current NAV) - created_at: DateTime with timezone, NOT NULL, default utcnow  Add
unique constraint on (investor_shares_id, valuation_date) to prevent duplicate valuations for
the same holding on the same date.  Add index on valuation_date for efficient querying of all
snapshots for a given month. Add index on investor_shares_id for efficient querying of all
```

```
historical valuations for a holding.  The model should go in
app/Valuation/valuation snapshot.py. Example record: investor shares id=1,
valuation_date=2026-04-10, nav_per_share=1389.5737, shares_held=771.92,
market_value=1072636.69, performance_pct=4.52
```

## Migration 5: Add columns to batches table

The existing batches table needs new columns to support auto-batching, share class linkage, and the new deployment workflow.

**AI PROMPT: Migration 5**

```
Create an Alembic migration that adds columns to the existing 'batches' table in the OFSD
Flask backend.  New columns to ADD (do NOT drop or rename existing columns): -
share_class_id: Integer, FK to share_classes.id, NULLABLE (null for legacy batches created
before migration) - is_open: Boolean, DEFAULT True, NOT NULL (True while batch is accepting
new investors, False after Thursday close) - auto_close_at: DateTime with timezone, NULLABLE
(set to next Thursday 9:00 AM EAT when batch is created) - exported at: DateTime with
timezone, NULLABLE (timestamp when the batch Excel was exported) - total_shares:
Numeric(20,6), NULLABLE (sum of all investor shares in this batch, populated at deployment) -
deployment_nav: Numeric(20,6), NULLABLE (NAV per share at the time this batch was deployed) -
deployment_date_actual: Date, NULLABLE (actual date Head Office deployed the funds, from
confirmation email)  IMPORTANT: share class id is NULLABLE because existing legacy batches
will not have a class assigned. New batches created through the auto-batch system will always
have a share class id.  Add index on (share class id, is open) for efficient lookup of open
batches per class.  The existing Batch model is in app/Batch/model.py. Add the new columns to
that model. Existing columns that STAY (do not modify): id, batch_name, certificate_number,
date_deployed, duration_days, is_active, stage, is_transferred, transfer_transaction_cost,
transfer_entry_fee_percent, entry_fee_percentage, created_at, updated_at.
```

## Migration 6: Add columns to investments table

The investments table needs a link to the share class so we know which class each investment is targeting.

**AI PROMPT: Migration 6**

```
Create an Alembic migration that adds columns to the existing 'investments' table in the OFSD
Flask backend.  New columns to ADD (do NOT drop or rename existing columns): -
share_class_id: Integer, FK to share_classes.id, NULLABLE (null for legacy investments
created before migration) - deposit date: Date, NULLABLE (the actual date the client
deposited money, as distinct from date_deposited which is the system entry timestamp. This
field is entered by the user on the Add Investor form.)  IMPORTANT: share class id is
NULLABLE because existing legacy investments will not have a class assigned. New investments
created through the Add Investor flow will always have a share_class_id.  The existing
Investment model is in app/Investments/model.py. Add the new columns to that model. Existing
columns that STAY (do not modify): id, investor_name, investor_email, investor_phone,
internal client code, amount deposited, batch id, fund id, fund name, date deposited,
date_transferred, wealth_manager, IFA, contract_note, valuation, transaction_fee_usd,
entry_fee_usd, main_balance.  After adding the columns, create a data migration step that
attempts to populate share_class_id for existing investments by matching fund_id to
core_funds, then finding the corresponding share_class. This may not work for all records if
share classes haven't been created yet, which is fine - those will be populated manually
during the backloading process.
```

## 5.2 Core Valuation Engine

This is the heart of the new system. Replace app/logic/valuation_service.py entirely.

### 5.2.1 Share Calculation (at deployment)

```
def calculate_investor_shares(deposit_amount, transfer_cost_share, entry_fee_pct,
nav_per_share):
    net_amount = deposit_amount - transfer_cost_share - (deposit_amount *
entry_fee_pct / 100)
    shares = net_amount / nav_per_share
    return shares  # Numeric(20,6) precision
```

### 5.2.2 Valuation (monthly)

```
def valuate_class(share_class_id, current_nav, total_fund_nav, valuation_date):
    holdings = InvestorShares.query.filter_by(share_class_id=share_class_id,
is_active=True)
    system_total = sum(h.shares_owned for h in holdings) * current_nav
    diff = abs(system_total - total_fund_nav)
    if diff > Decimal('1.00'):
        raise ReconciliationError(f'Mismatch: system={system_total},
HO={total_fund_nav}')
    for h in holdings:
        market_value = h.shares_owned * current_nav
        perf = (current_nav - h.deployment_nav) / h.deployment_nav * 100
        # Save valuation_snapshot
```

### 5.2.3 The Math (Reference from NAV Statement)

Using real data from the AXYS NAV statement dated 10 April 2026:

| Class | Shares | Prev NAV | Current NAV | Performance | Market Value |
|---|---|---|---|---|---|
| KES Class I | 771.92 | 1,329.4779 | 1,389.5737 | 4.52% | 1,072,636.69 |
| USD Class I | 14,421.80 | 14.4809 | 15.1905 | 4.90% | 219,074.79 |
| USD Class R | 80.30 | 12.3327 | 12.9358 | 4.89% | 1,038.81 |

**Verification formulas:**

```
Market Value = Shares x Current NAV
  771.92 x 1,389.5737 = 1,072,636.96 (statement shows 1,072,636.69, diff = 0.27
from rounding)
Performance = (Current NAV - Prev NAV) / Prev NAV x 100
  (1,389.5737 - 1,329.4779) / 1,329.4779 x 100 = 4.52%


Total KES = 1,072,636.69
Total USD = 219,074.79 + 1,038.81 = 220,113.60
```

**DECIMAL PRECISION**

NAV per share uses 4-6 decimal places. Shares use up to 6 decimal places. Market values round to 2 decimal places. Small rounding differences (< 1.00 in currency) between system and AXYS are expected and acceptable. The reconciliation tolerance should be configurable.

**AI PROMPT: New Valuation Service**

```
Rewrite the valuation service for the OFSD Flask backend. The current service is at
app/logic/valuation_service.py and uses percentage-based compounding. Replace it with NAV-
based valuation.  Core logic: 1. Input: valuation_date, list of (share_class_id,
nav_per_share, total_fund_nav) 2. For each class: load all active investor shares records 3.
Calculate system_total = sum(shares_owned) * nav_per_share 4. Compare to total_fund_nav. If
abs(diff) > 1.00: raise error 5. For each investor holding: market_value = shares_owned *
nav_per_share 6. performance_pct = (nav_per_share - deployment_nav) / deployment_nav * 100 7.
Save valuation snapshot record for each holding 8. Save nav_history record for each class
Use Decimal throughout for financial calculations. Never use float. The reconciliation
tolerance of 1.00 should be a configurable constant. NAV per share should support up to 6
decimal places (Numeric(20,6)). Market values should be 2 decimal places (Numeric(20,2)).
```

## 5.3 Auto-Batch Logic

**AI PROMPT: Auto-Batch Service**

```
Create a service in the OFSD Flask backend that handles automatic batch creation and closure.
When a new investor is added (POST /api/v1/investors): 1. Look for an open batch with
matching share_class_id where is_open=True and auto_close_at > now() 2. If found: add
investor to that batch 3. If not found: create new batch with:    - batch_name: auto-
generated as '{fund_code} {class_code} {next_thursday_date}'    - share_class_id: from the
investor's selected class    - is_open: True    - auto_close_at: next Thursday 9:00 AM EAT
(UTC+3) 4. Add investor to the new batch  Batch auto-close (scheduled task or cron): - Every
Thursday at 9:00 AM EAT: find all batches where is_open=True and auto_close_at <= now() - Set
is_open=False, generate export Excel - If no investors in batch, delete it instead  CRITICAL:
A batch belongs to exactly ONE share class. Two investors in different classes cannot share a
batch.
```

## 5.4 Withdrawal Processing

Withdrawals redeem shares at the current NAV. The process has its own tracker:

1. Request Received: Record withdrawal request, calculate estimated shares to redeem
2. Initiated: Sent to Mauritius for processing
3. Confirmation: Mauritius provides redemption NAV and shares redeemed
4. Transfer: Funds being sent back to Kenya
5. Completed: Funds credited, investor shares updated, statement generated

**Exit fee rule:** If the investor joined the fund less than 6 months ago (from deployment_date to withdrawal_date), a 2% exit fee is charged on the withdrawal amount.

## 5.5 Statement Generation

**AI PROMPT: Statement Generator**

```
Update the PDF statement generator for the OFSD system. Three statement types:  1. DEPOSIT
STATEMENT (generated at deployment):    - Header: Investor name, client code, fund name,
share class, currency    - Body: Deposit amount, transfer cost, entry fee (2%), net invested,
NAV per share at deployment, shares purchased, contract note number    - Footer: Date,
disclaimer  2. MONTHLY VALUATION STATEMENT:    - Header: Same as deposit    - Body: Shares
held, Previous NAV, Current NAV, Performance %, Current Market Value    - Historical
valuations table: Date | NAV | Value (one row per past valuation)    - Historical
transactions table: Date | Type | Amount (deposits and withdrawals)  3. WITHDRAWAL STATEMENT:
- Header: Same    - Body: Shares redeemed, Redemption NAV, Gross proceeds, Transfer cost,
Exit fee (if <6 months), Net withdrawal    - Remaining position: Shares remaining, Current
value  Use the existing PDF generator pattern at app/utils/pdf_generator.py. All monetary
values must show correct currency (KES or USD) based on the share class.
```

# 6. Testing & Validation

## 6.1 Reference Test Data (from AXYS NAV Statement - 10 April 2026)

Use this real data to validate every calculation in the system:

| Test | Input | Expected Output | Formula |
|---|---|---|---|
| KES Market Value | 771.92 shares x 1389.5737 NAV | 1,072,636.69 (accept 1,072,636.96) | shares x nav |
| KES Performance | Prev 1329.4779, Curr 1389.5737 | 4.52% | (curr-prev)/prev x 100 |
| USD-I Market Value | 14421.80 shares x 15.1905 NAV | 219,074.79 | shares x nav |
| USD-I Performance | Prev 14.4809, Curr 15.1905 | 4.90% | (curr-prev)/prev x 100 |
| USD-R Market Value | 80.30 shares x 12.9358 NAV | 1,038.81 | shares x nav |
| USD-R Performance | Prev 12.3327, Curr 12.9358 | 4.89% | (curr-prev)/prev x 100 |
| Total KES | Sum of KES classes | KES 1,072,636.69 | sum |
| Total USD | Sum of USD classes | USD 220,113.60 | sum |

## 6.2 Simulation Script

Before touching the main codebase, run this standalone validation:

**AI PROMPT: Simulation Script**

```
Write a standalone Python script (no Django/Flask dependencies) that validates the share-based valuation model.  Use this real data from the AXYS NAV statement (10 April 2026): Classes: - KES I: 771.92 shares, prev NAV 1329.4779, curr NAV 1389.5737, expected value 1,072,636.69 - USD_I: 14421.80 shares, prev NAV 14.4809, curr NAV 15.1905, expected value 219,074.79 - USD R: 80.30 shares, prev NAV 12.3327, curr NAV 12.9358, expected value 1,038.81 The script should: 1. Calculate market_value = shares x current_nav for each class 2. Calculate performance = (current nav - prev nav) / prev nav x 100 3. Compare calculated values to expected values 4. Show the difference (should be < 1.00 for values, < 0.01% for performance) 5. Test the reconciliation: sum(shares) x nav should equal total fund NAV 6. Test a deployment scenario: if someone deposited KES 1,000,000 with 2% entry fee and KES 150 transfer cost, at NAV 1329.4779, how many shares do they get? 7. Use Decimal for all calculations, never float  Print PASS/FAIL for each test with the actual vs expected values.
```

## 6.3 Integration Test Checklist

| Test | Steps | Expected Result |
|---|---|---|
| Create Fund | POST fund, POST 3 classes (KES_I, USD_I, USD_R) | Fund with 3 classes, correct currencies |
| Add Investor (KES) | POST investor with KES class | Auto-batch created, investor in batch |
| Add Investor (USD) | POST investor with USD class | Separate batch (different class) |
| Close Batch | POST close (or wait for Thursday) | Batch closed, Excel generated |
| Deploy Batch | PATCH deploy with NAV data | Shares calculated per investor |
| Recon Check | Compare system shares x NAV to AXYS | Difference < 1.00 |
| Monthly Valuation | POST new NAV for all classes | All investors updated, snapshots saved |
| Generate Statement | GET statement for investor | Correct shares, NAV, value, performance |
| Withdrawal | POST withdrawal request, confirm | Shares reduced, statement generated |

# 7. Entity Relationship Summary

This section maps how all entities relate to each other in the new system.

```
CoreFund (1) ---> (many) ShareClass
ShareClass (1) ---> (many) Batch       [each batch belongs to exactly one class]
ShareClass (1) ---> (many) NavHistory   [NAV tracked per class per date]
Batch (1) -------> (many) Investment    [investors in the batch]
Investment (1) --> (many) InvestorShares [shares per class per batch]
InvestorShares (1) -> (many) ValuationSnapshot [monthly valuations]
```

**Key constraints:**

- A ShareClass has exactly ONE currency (KES or USD). This is enforced at database level.
- A Batch belongs to exactly ONE ShareClass. Two investors in different classes cannot share a batch.
- An investor can have multiple InvestorShares records (different classes, different batches, different deposits).
- NavHistory has one record per class per date per source. No duplicates.
- ValuationSnapshot has one record per InvestorShares per valuation date.

# 8. Glossary

| Term | Definition |
|---|---|
| NAV | Net Asset Value. The total value of all assets in a fund or class. |
| NAV per Share | The price of one share in a class. Market value = shares x NAV per share. |
| Share Class | A subdivision of a fund with its own NAV and currency. E.g. KES Class I, USD Class I, USD Class R. |
| Deployment | When investor funds are actually invested by AXYS. This is when shares are purchased. |
| Deployment NAV | The NAV per share on the date of deployment. Used to calculate initial shares. |
| Reconciliation | Checking that system-calculated totals match AXYS figures. |
| Batch | A weekly grouping of investors (Friday to Thursday) sent to AXYS together. |
| Contract Note | Official document from AXYS confirming deployment details. |
| Entry Fee | 2% fee charged on deposit amount before shares are purchased. |
| Exit Fee | 2% fee charged on withdrawal if investor exits within 6 months of joining. |
| Transfer Cost | Bank transfer fees for moving money to Mauritius. |
| AXYS | The offshore fund manager in Mauritius that manages the actual fund. |